

Constraint and Clue Model

Turn copied clue cards into a deterministic rule result

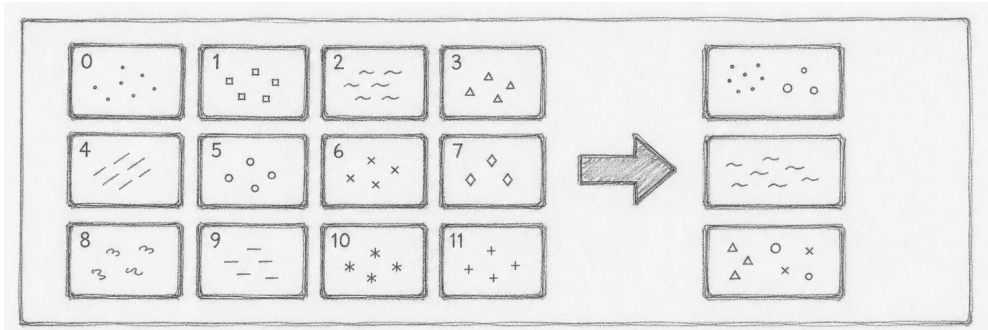
E0 COPIED POLICY REPLAY — ZERO HARDWARE

Lesson	055 — component	Time	150–210 minutes
Prerequisite	copied evidence and modular time	Board	compile-only Mega replay
Capacity	12 clues, 12 rules	Observation	named memory result cells

Responsibility. `ClueConstraintModel` owns one fixed rule graph and copied clue evidence. It validates source identity, sequence, time, quality, and prerequisites before publishing masks and a model disposition. It owns no endpoint, pin, clock, display, storage medium, latch, lamp, lock, or physical room.

1 Begin with an evidence wall, not a magic answer

Each clue has one dense ID from zero, one fixed source identity, and one matching slot and mask bit. An update is an entire envelope: selected slots must contain matching clue IDs, while unselected slots must be canonical zero. A malformed slot rejects the whole update rather than mixing old and new evidence.

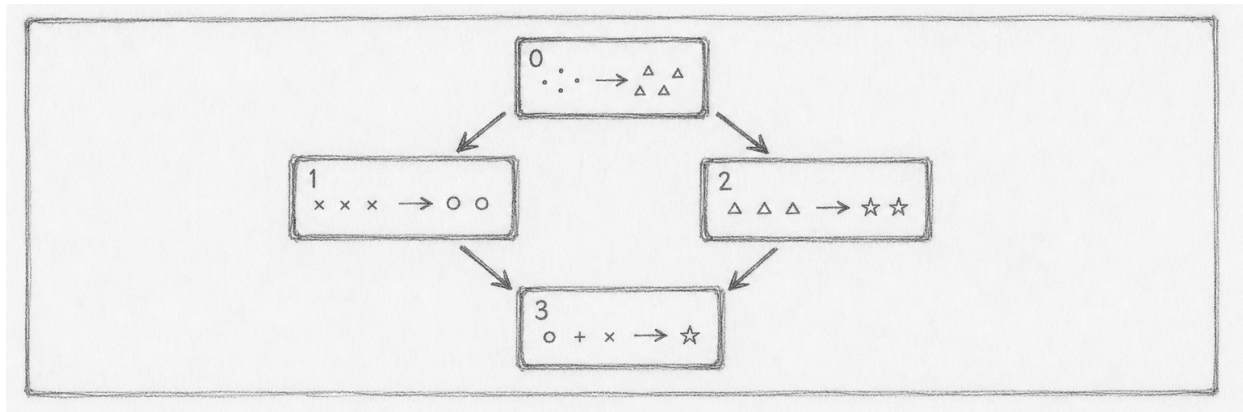


Alternative text: a visibly hand-drawn wall of twelve clue cards records source, sequence, time, and quality. One sketched arrow leads to satisfied and blocked rule masks plus a model disposition.

This is a conceptual memory drawing, not a circuit layout or formal schematic. The “wall” is an array in memory. The named observation place for this E0 lesson is `evidenceWallCells[]` in the canonical sketch.

2 Build a directed acyclic rule graph

A rule contains at most four terms and four prerequisite rule IDs. A term compares one clue category with `Equals` or `NotEquals`. Prerequisites are evaluated first. IDs are dense and unique; duplicate terms, duplicate prerequisites, self-edges, out-of-range IDs, noncanonical unused fields, and any cycle make initialization fail.



Alternative text: a graphite diamond graph has rule zero feeding rules one and two, which both feed rule three. Rule three cannot be satisfied before both prerequisites.

The model compiles this graph to a stable iterative order during `initialize()`. It uses no recursion, allocation, callback, hidden clock, or container iteration order.

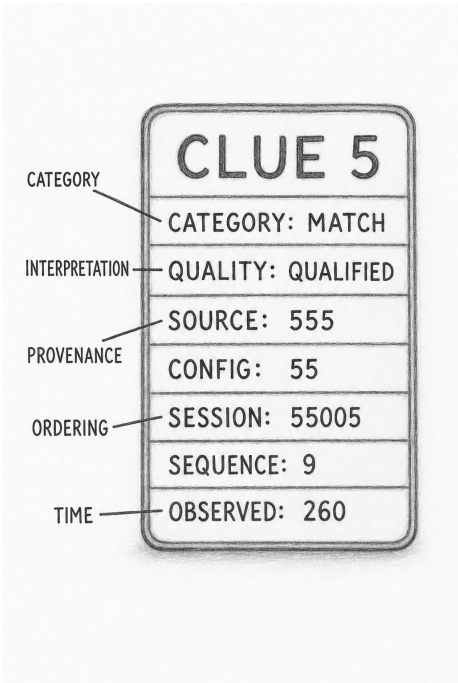
3 Predict evaluation precedence

For a structurally valid envelope, evidence is classified before term matching. Source, timing, invalid quality, or failing status is invalid; stale precedes contradictory; contradictory precedes missing. A category mismatch is ordinary unsatisfied evidence. For each rule, a failed prerequisite precedes its terms.

Observed condition	Rule prediction	Model consequence
qualified, fresh, matching category	term may satisfy	continue through DAG
qualified, fresh, different category	Unsatisfied	incomplete
required clue absent	MissingEvidence	incomplete
invalid source, time, quality, or status	InvalidEvidence	invalid evidence
older than inclusive age limit	StaleEvidence	stale evidence
explicit contradiction	ContradictoryEvidence	contradiction
failed prerequisite	BlockedByPrerequisite	blocked mask bit set
all rules satisfied	Satisfied	solved

4 Keep provenance attached to every cell

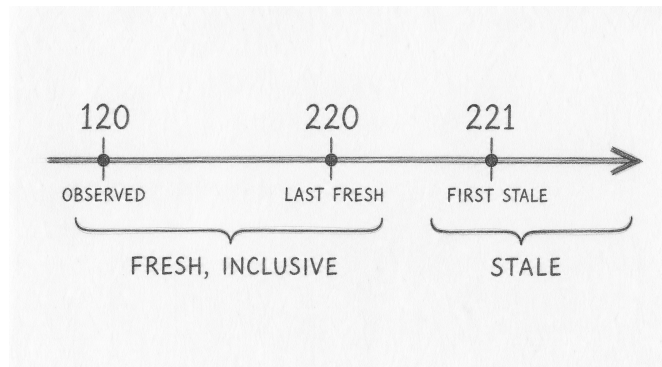
An accepted clue retains category, quality, exact expected source ID, configuration revision, session epoch, source sequence, observation time, and complete status. A changed value at the same sequence is invalid. Forward sequence including rollover is accepted; regression and exact-half-range ambiguity reject. Update-array order never supplies precedence.



Alternative text: a sketched clue-five record lists category, quality, source identity, configuration, session, sequence, and time. Pencil leader lines label category, interpretation, provenance, ordering, and time as inseparable evidence.

5 Treat freshness as an inclusive interval

The caller supplies `now`; the model never reads wall time. With `maximumEvidenceAge = 100`, evidence observed at 120 is fresh through 220 and stale at 221. A future timestamp, regression, duration at or above the unsigned half range, or ambiguous order rejects atomically.



Alternative text: a rough pencil timeline marks observation at 120, the inclusive last-fresh point at 220, and the first-stale point at 221.

6 Read the canonical code path

The full sketch remains downloadable and testable as text. Its high-level flow deliberately reads as observe, decide, present:

```
const ClueConstraintUpdate update =
  observeCopiedClueCards(frame);
const Status operationStatus =
  decideClueProgress(update);
presentEvidenceWallIntent(replayIndex, frame,
  operationStatus);
```

Each frame is one copied envelope. The sketch first proves that a four-rule cycle is rejected, then replays four-rule progress and solve, a maximum twelve-rule solve, stale evidence, and contradictory evidence. It changes from the four-rule object to the twelve-rule object only at a deliberate lifecycle boundary.

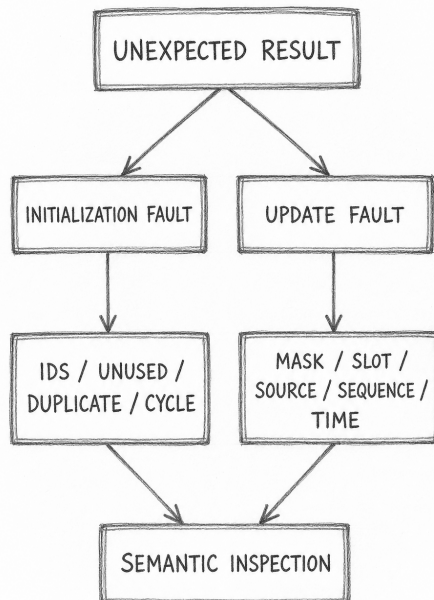
7 Run predict, observe, interpret

Use the canonical compile-only Mega sketch and focused host tests. Do not credit Serial output. Before each frame, write a prediction; afterward inspect the named memory cell or fieldwise test result.

1. **Cycle fixture.** Predict `InvalidConfiguration`. Observe `cycleRejectionStatusCell` and `cycleRejectionDispositionCell`. Interpret success only as graph validation.
2. **Four-rule progress.** Predict satisfied mask `0x0001`, blocked mask `0x000e`, and `Incomplete`. Observe `evidenceWallCells[0]`.
3. **Four-rule solve.** Predict `0x000f`, `0x0000`, and `Solved`. Observe cell 1.
4. **Maximum solve.** Predict all twelve bits satisfied, no blocked bits, and `Solved`. Observe cell 2.
5. **Stale frame.** Predict stale disposition and blocked downstream rules. Observe cell 3 and its copied clue-zero time.
6. **Contradiction.** Predict the first five rules satisfied, the contradictory clue and its descendants blocked, and a contradiction disposition. Observe cell 4.

frame	generation	satisfied	blocked	disposition / pass
progress	_____	_____	_____	_____
four solved	_____	_____	_____	_____
twelve solved	_____	_____	_____	_____
stale	_____	_____	_____	_____
contradiction	_____	_____	_____	_____

8 Diagnose configuration and update faults



Alternative text: a hand-drawn troubleshooting tree separates initialization faults in IDs, unused fields, duplicates, and cycles from update faults in mask/slot identity, provenance, sequence, and time. Accepted updates then lead to semantic quality, freshness, prerequisite, and term inspection.

Symptom	Inspect first	Do not conclude
initialization rejects	dense IDs, canonical unused fields, DAG	bad sensor
update rejects	every selected and unselected slot	partial clues were copied
invalid evidence	source, sequence, time, quality, status	puzzle is false
blocked rule	first blocking prerequisite	its own term failed
unsatisfied rule	first blocking term and category	input is electrically bad
solved	all rule snapshots and provenance	person, access, or room is safe

9 Separate acquisition and safe-state evidence

Acquisition prediction. A canonical configuration with 1–12 clues, 1–12 acyclic rules, valid fixed source identities, bounded age, and canonical unused fields initializes.

Acquisition observation. Host tests and named memory cells inspect initialization status, lifecycle state, configuration identity, compiled rule results, and copied evidence. No hardware resource is acquired.

Acquisition interpretation. Success proves bounded configuration and deterministic E0 policy only. It does not prove a pin, input voltage, debounce circuit, display, storage device, latch, or Mega execution.

Safe-state prediction and observation. Construction, failed initialization, reset, shutdown, invalid envelopes, and lifecycle exhaustion cannot publish physical actuation because this component has no actuation path. Inspect empty or inert snapshots and unchanged retained evidence where the operation is atomic.

Safe-state interpretation. Software state is not an emergency stop or power isolation. No person or animal may depend on this model for access, confinement, egress, alarm, or life safety.

10 Record resource evidence without overclaiming

The promotion gates for the standalone maximum are 16/20 KiB flash, 1,536/2,048 bytes static SRAM, 384/512 bytes synchronous stack, and 512/640 bytes for the reusable object. The final canonical no-LTO Mega 2560 build measures 8,886 bytes of flash and

1,261 bytes of static SRAM. `sizeof(ClueConstraintModel)` is 636 bytes. Conservative compiler-derived synchronous stack evidence is 412 bytes.

Evidence	Measured	Gate	Disposition
flash	8,886 B	16/20 KiB	target and hard gate pass
static SRAM	1,261 B	1,536/2,048 B	target and hard gate pass
reusable object	636 B	512/640 B	reviewed target miss; hard gate passes
synchronous stack	412 B	384/512 B	reviewed target miss; hard gate passes

These values are toolchain evidence, not runtime high-water measurement or physical verification. A target miss requires review; a hard-ceiling miss stops promotion. The object and stack target misses were reviewed and retained because both hard ceilings pass; they do not authorize growth past those ceilings. Re-measure after any source, LTO setting, flags, compiler, core, fixture, or composition change. For a local reproduction, record: **flash** _____, **SRAM** _____, **object** _____, and **stack** _____.

11 Keep E1 and E2 closed

Lesson 055 is E0. There is deliberately no wiring table, formal schematic, or bench circuit. E1 would require exact passive input and presentation specimens, primary electrical evidence, voltage and polarity limits, resource ownership, an authoritative schematic, non-Serial observation, rollback, and recorded bench acceptance. Powered actuation remains absent.

Any later E2 demonstration belongs to Lesson 057's separate gate and may use only a restrained demonstration servo or inert current-limited relay/lamp load with independent physical load-power removal. It may never attach to a door, lock, occupied enclosure, alarm, emergency lighting, egress route, or life-safety system.

12 Exercises and acceptance record

1. Change one selected slot's clue ID while preserving its mask bit. Predict the operation status and prove the prior snapshot is unchanged.
2. Add one cycle edge to the four-node diamond. Identify the exact configuration rule violated.
3. Put a qualified observation exactly at the freshness limit, then one tick beyond it. Compare the rule and model dispositions.
4. Change one source field, then change one value at the same sequence. Explain why both are evidence faults rather than ordinary mismatches.
5. Construct a valid **NotEquals** term and predict its first blocking term for both categories.

Record	Result	
cycle rejection	status _____	disposition _____
acquisition	status _____	initialized _____
maximum replay	satisfied _____	blocked _____
freshness boundary	last fresh _____	first stale _____
atomic rejection	retained generation _____	unchanged _____
safe-state result	_____	
reviewer / date	_____	/ _____

This is a deterministic policy worksheet, not a hardware acceptance record. Compare it with the accessible HTML reference, the canonical Lesson 055 sketch, and the focused host tests. Lesson 056 will compose copied controls, presentation, stop, acknowledgement, and a two-slot audit image without turning this clue result into access authority.